JavaScript 1.6

Collections, Symbols & Iteration

Print-first interview refresher | keys → ownership → iteration

Trace three things:
key equality | lifetime
who advances the cursor?

1 | Choose by key and operation

Plain object
Named fields; string or symbol keys
Object.keys: own enumerable strings

Map
Any key; pairs in insertion order
has distinguishes missing; size counts

Set
Unique values; insertion order
has tests membership; size counts

SameValueZero: NaN matches NaN; +0 and -0 match; objects match by identity.
Update keeps position. Delete + reinsert moves to the end. Deduplicate records by a stable ID.

2 | Cross the JSON boundary explicitly

```
JSON.stringify(new Map([["id", 7]])) → "{}"
```

Encode entries for JSON-safe keys. Object.fromEntries can merge 7 and "7". Object identity needs a stable ID.

3 | Choose who owns the lifetime

WeakMap
Attach data by identity.

WeakSet
Mark an instance as seen.

WeakRef
deref() → target or undefined.

Objects and non-registered symbols can be weak targets. WeakMap and WeakSet have no iteration or size.
GC timing is unspecified. Use an explicit fallback and policy if a cache must be bounded.

4 | Symbols are keys and language hooks

Symbol("id")
New identity on each call.

Symbol.for("id")
Shared identity from the registry.

Well-known symbols
Hooks such as Symbol.iterator.

Reflect.ownKeys sees symbol keys. Two Symbol() calls stay distinct even with the same description.

5 | Iterate on demand

`Iterable` opens a traversal → `[Symbol.iterator]()` returns a cursor → `next()` advances it → `{ value, done }` item or completion

A fresh cursor makes traversal repeatable. A generator is a one-shot cursor that can iterate itself.
yield delivers an item. return ends the sequence. yield* delegates to another iterable.
Early exit from for...of calls return() if present. A generator can release resources in finally.
Iterator helpers are lazy. Bound the source with take before collecting via toArray.

6 | Await the next result

`[Symbol.asyncIterator]()` opens an async cursor → `await next()` waits for the next result → `for await...of` consumes and can exit early

An async generator can await a page and yield it. Early exit awaits return() if present.
Array.fromAsync materializes a finite source. Iterate directly for a prefix or incremental rendering.

Interview check
Choose identity or stable IDs. Decide if a traversal is repeatable. Say who closes the cursor.